Performance Testing Linked Lists

Kevin Browne

Why use a linked list?

- A linked list can be ***faster*** than a dynamically allocated array in some instances and have ***more consistent performance*** too!

- Let's run a test to compare performance!
  - Compare deleting head node 10,000 times to re-allocating and copying memory for an array 10,000 times to "delete" the first array element
  - This test is a ***bit*** contrived and unrealistic of course, but we're doing this to demonstrate a point regarding relative performance

Deleting first element in an array

```
int num_elements = 10000;
clock_t tic, toc;
printf("\n\n");

int *array = malloc(num_elements * sizeof(int));
for (int i = 0; i < num_elements; i++) array[i] = i;

tic = clock();
for (int i = 1; i < num_elements; i++)
{
  int *newarray = malloc((num_elements - i) * sizeof(int));
  for (int j = 0; j < num_elements - i; j++)
  {
    newarray[j] = array[j + i];
  }
  free(array);
  array = newarray;
}
toc = clock();
printf("Elapsed: %f seconds\n", (double)(toc - tic) / CLOCKS_PER_SEC);
free(array);
```

Deleting the head node in a list

```
Node *listp = NULL;
for (int i = 0; i < num_elements; i++) listp = insert_at_head(listp,i);

tic = clock();
for (int i = 0; i < num_elements; i++) listp = delete_at_head(listp);
toc = clock();
printf("Elapsed: %f seconds\n", (double)(toc - tic) / CLOCKS_PER_SEC);
```

We get that the linked list performance (bottom) easily beats the array performance (top)...

```
Elapsed: 0.234219 seconds
Elapsed: 0.000210 seconds
```

Though it's a contrived situation, these performance differences play out in real applications, motivating the use of data structures like linked lists.